HOTMOBILE 2027 · SUBMISSION DRAFT

μKV: Sequence-Level, Attention-Scored KV-Cache Eviction That Pays on a Phone

An LLM keeps the whole conversation in a cache a phone cannot afford, so eviction throws parts of it away. We ran nine published methods on a real phone and measured speed, energy and heat. One method ran 3.3x faster on the phone's CPU and 3.7x slower on its GPU. The hardware, not the method, decides whether eviction is worth doing.

WHAT WE CONTRIBUTE

The first end-to-end account of published eviction on a phone, measured on speed, energy, temperature and cache actually freed, plus the three device-level failures that decide whether it pays.

4.80x
faster than the full cache, phone CPU

7.4%
of the cache still live, and really freed

-58%
energy for the same 4096 tokens

Md Romyull Islam · Kennesaw State University · OnePlus 15, Snapdragon 8 Elite Gen 5

Eviction is designed on server GPUs and justified by one number

What the literature reports

How much cache it drops, a nominal budget K

Quality on server GPUs (LongBench, NIAH)

Per-head keep-sets, paged KV, unlimited power

KeyDiff comes closest: scoring-kernel latency on a phone, not end-to-end

What a phone deployment needs

Cells **actually freed**, not the budget requested

Throughput **by phase**, because prefill and decode differ

Energy and **temperature**: the device throttles

Whether the engine can even **execute** the policy

The systems that do run on phones (KVSwap, DynaKV, MobiLoRA) move bytes rather than discard them. None evicts. None reads the battery.

SnapKV asks for 1024 cells and keeps 8893.

THE PROBLEM

Every policy reports how much cache it drops. For the four policies that choose per head, that number is not what gets freed on this phone. For the two that choose at sequence level, it is.

WHY IT HAPPENS

Phone engines store the KV cache in ONE array shared by all heads and layers. SnapKV, Ada-KV, H2O and TOVA each pick a different keep-set per head. The engine can only free a cell that EVERY head agreed to drop, so the keep-sets pile up and almost nothing is released.

OUR SOLUTION

μKV picks ONE keep-set for all heads and layers. There is nothing to union, so every cell it drops is actually freed and the survivors can be packed together. KeyDiff is sequence-level too, so its budget is also real; we differ in what happens next, not here.

THE EVIDENCE

91.3%

SnapKV still holds 91.3% of the cache after asking to keep 1024 of 9741 cells. It pays the cost of eviction and collects none of the benefit.

The split is per-head vs sequence-level, not us vs everyone

Llama-3.2-1B (8 KV heads), phone CPU, 9741-token prompt

Policy	selects	asked	live cells	retained
SnapKV	per head	1024	8893	91.3%
H2O	per head	20% of N	9151	94%
TOVA	per head	2048	4093	42.0%
Ada-KV	per head	2048	3489	35.8%
KeyDiff	sequence	2048	2049	21.0%
µKV	**sequence**	**1024**	**721**	**7.4%**

Read the two right-hand columns together. The four per-head policies get back far more cache than they asked for. The two sequence-level policies get almost exactly what they asked for.

KeyDiff asked for 2048 and kept 2049. Its budget is real, so it is NOT a victim of this problem. That is why it is absent from the failure list.

AND IT GETS WORSE WITH SIZE

Llama-1B has 8 KV heads. Phi-3 has 32. With four times as many heads, unanimity gets rarer, and the whole per-head class collapses into one band:

85 to 100%

retained on Phi-3. H2O logs 20.4 million eviction attempts that free zero cells. As models get wider, per-head eviction becomes a no-op that still costs energy.

Reading attention scores breaks the GPU output

THE PROBLEM

To score which cells matter, a policy must read the attention weights. The normal way is an eval-callback on the softmax node. On the Adreno GPU, doing this makes the model emit garbage tokens.

WHY IT HAPPENS

A callback changes how the graph is EXECUTED, not how it is split across backends: llama.cpp runs the work as fragments with a sync after each. The CPU backend handles that correctly. On this phone GPU it does not.

OUR SOLUTION

μKV never intercepts. It adds a small extra node (kq_evict) INSIDE the graph, so the scores are something the graph produces on its own. No callback, no corruption, and FlashAttention stays on the whole time.

THE EVIDENCE

76 / 609

degenerate tokens once a callback reads the tensor. With no callback the same run gives 0 of 978.

Two controls, each removing one suspect

Control 1: is turning FlashAttention off the cause?

Same run, FA off	degenerate
vanilla, no callback	0 / 978
policy, callback reads tensor	76 / 609
policy, callback never reads	68 / 679

No. All three runs are vanilla llama.cpp with FA off. Only the ones with a callback corrupt.

Control 2: is it the phone, or the GPU specifically?

Needle retrieval	CPU	Adreno
H2O (uses callback)	8/14	3/14
TOVA (uses callback)	12/14	2/14
µKV (no callback)	**13/14**	**14/14**

The GPU. Across all callback cells: 0.3% corrupt tokens on CPU, 13.9% on Adreno.

The mechanism, from the llama.cpp scheduler source

In ggml_backend_sched_compute_splits the no-callback path computes the whole split in one call; the callback path builds ggml_graph_view fragments and synchronizes after each. The chunking is decided when the callback is ASKED for a node, before any read, which is why requesting without reading corrupts the same. The same path is correct on the CPU backend. We did not test desktop Vulkan, so we do not separate llama.cpp's Vulkan code from the Adreno driver; we report only that this route is unsafe on this device.

The same policy is 4.80× on one processor and 1.20× on the other

THE PROBLEM

Papers report one speedup number per policy. On a phone that number is not a property of the policy at all. It changes by a factor of four depending on which processor runs it.

WHY IT HAPPENS

Every decoded token reads the model weights W plus the live cache KV. Eviction can only shrink KV. So even a zero-size cache cannot go faster than (W+KV)/W. On the phone GPU, Llama-1B weights are 800 MB against a 319 MB cache, so that ceiling is 1.40×, and no evictor

OUR SOLUTION

We report the bound next to every speedup, on both processors, and we check whether it actually binds. On the CPU the bottleneck is attention compute over live cells, not weight bytes, so the roofline does not apply at all and the same μKV runs 4.80x.

THE EVIDENCE

1.40×

is the analytical upper bound for ANY evictor on Llama-1B / phone GPU, and none exceeded it. But μKV at 7.4% retention and StreamingLLM at 20.6% both land at 1.20x, so on this backend something saturates before the cache term does.

Why we report the model and the processor, never a bare speedup

Model and processor	weights W	cache KV	ceiling	µKV got
Llama-1B · phone GPU	800 MB	319 MB	1.40×	1.20×
Phi-3 · phone GPU	smaller	bigger	2.83×	2.64×
Llama-1B · phone CPU	**not the limit**	**not the limit**	**no ceiling**	**4.80×**

The third row is the one that answers 'why no number for Llama on CPU'. The ceiling formula assumes the device is waiting on weight bytes. On the CPU it is not: it is waiting on attention arithmetic over the cells that are still live. Shrink the cells and you shrink the work directly, so there is no roofline to hit and the same policy reaches 4.80×.

Best proof is a rival, not us

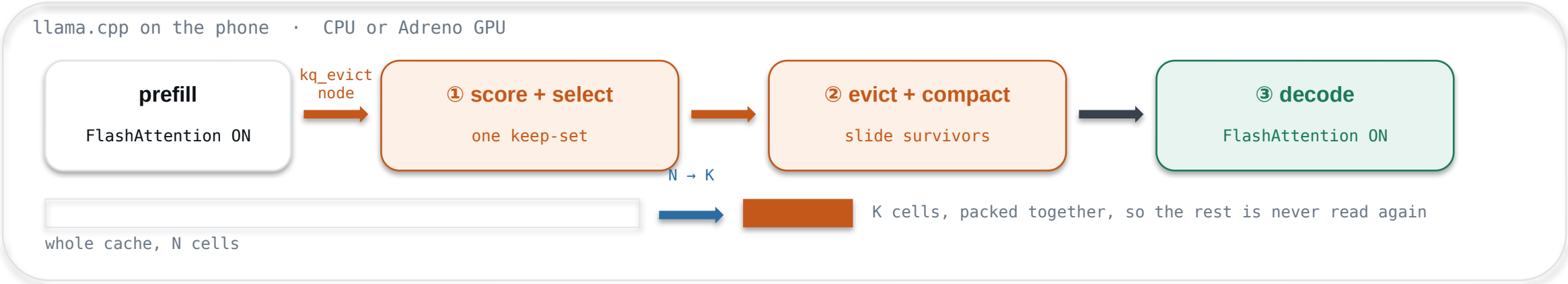
KeyDiff, same model (Llama-3.2-1B), same budget, same prompt:

3.29× phone CPU

~0.06× phone GPU

On the GPU it is about sixteen times SLOWER than keeping the whole cache. Nothing about the policy changed. Only the processor. The GPU point is a single run; vanilla at that prompt spans 24.1-28.6 tok/s, so the ratio is 0.056 to 0.067.

Each of the four decisions answers one of the three breaks



④ thermal watchdog reads the sensors and lowers the clock BEFORE the vendor cliff — it never touches the keep-set

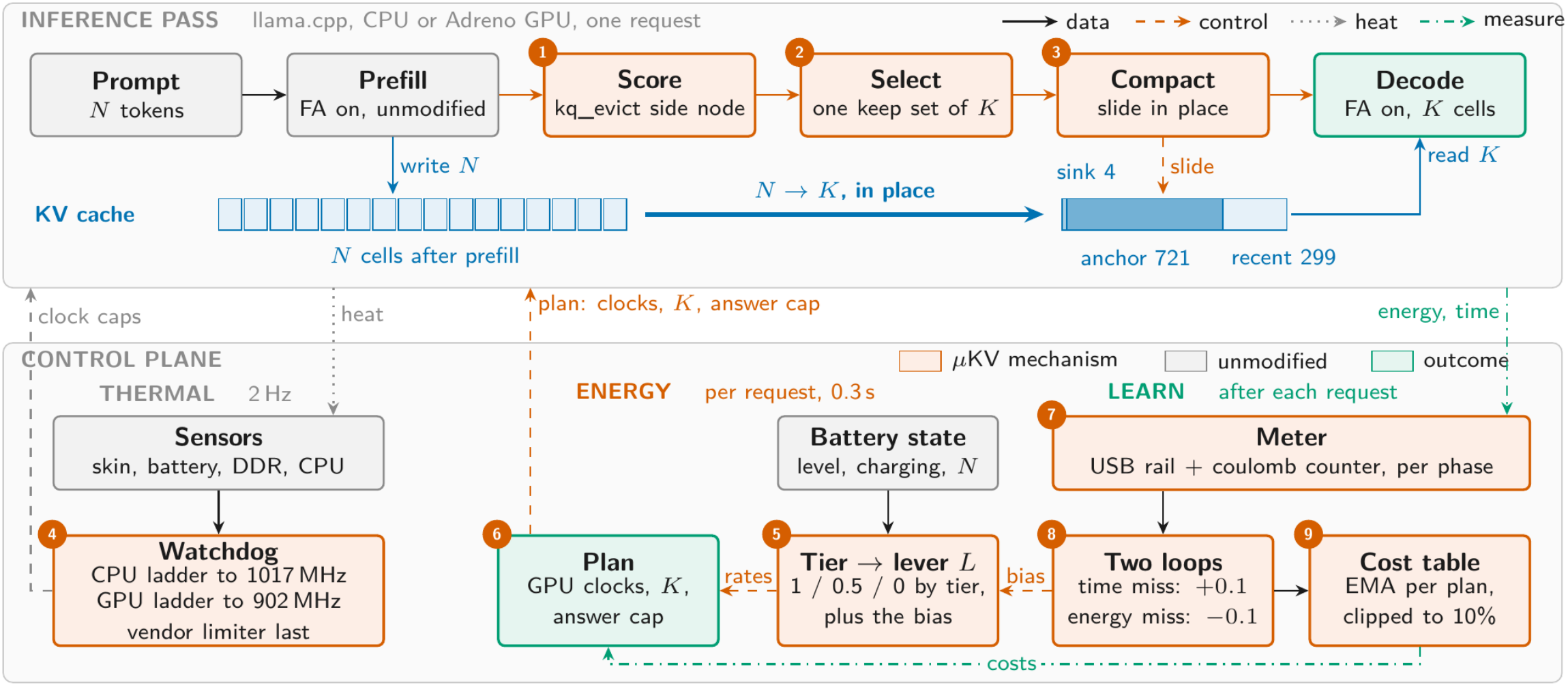
① score inside the graph — answers BREAK 2. The scores come from a node the graph produces, so no callback and no corruption.

② one keep-set, packed — answers BREAK 1. One set for all heads means every dropped cell is really dropped, and packing them makes the survivors contiguous.

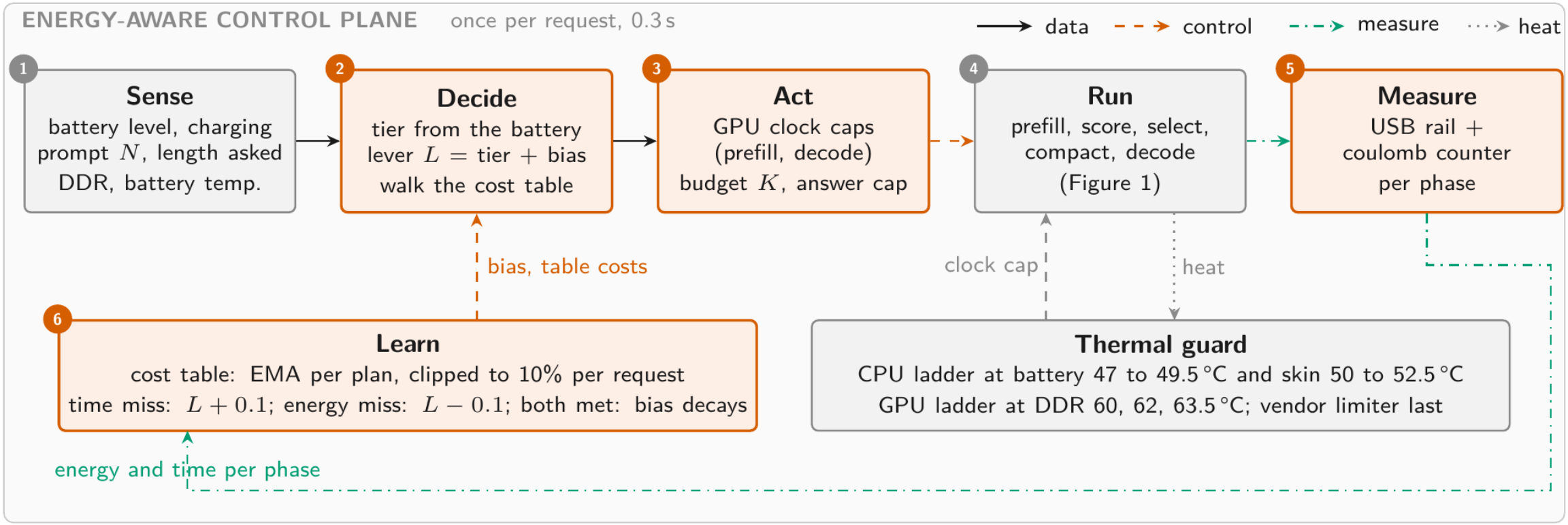
③ freeze during decode — the keep-set is chosen once at the end of prefill and never changes. Only the recent window slides. So there is no per-token scoring cost.

④ compact in place — the survivors are slid down inside the tensors the engine already has. No second copy of the cache is ever allocated.

THE PASS ON TOP, THE CONTROL PLANE BELOW



SENSE, DECIDE, ACT, RUN, MEASURE, LEARN: ONCE PER REQUEST



4.80× faster than the full cache, on one twelfth of the cells

Llama-3.2-1B Q4_K_M · phone CPU, 6 threads · 9.7K-token prompt + 4096 decoded tokens

Policy	config	tok/s	×van	wall (s)	mWh	live cells
µKV	**K=1024**	**23.91 ±0.37**	**4.80×**	**379**	**595**	**721**
KeyDiff	K=2048	16.40 ±0.11	3.29×	484	700	2049
Ada-KV	FA on	11.52	2.29×	566	845	3489
StreamingLLM	4+2000	10.25 ±0.06	2.06×	580	781	2001
SnapKV	FA off	6.83	1.36×	849	1180	5931
H2O	FA off	6.45	1.29×	856	1262	9151
SnapKV	FA on	5.38	1.07×	953	1346	8893
vanilla	full cache	4.99 ±0.01	1.00×	1004	1431	9741

How to read this: the last column is what the engine still holds. The policies at the bottom are slow BECAUSE they never freed anything.

4.80×

23.91 vs 4.99 tok/s, three runs each, rotated order

−58%

energy for the same 4096 tokens, 595 vs 1431 mWh

12.3×

fewer live cells than SnapKV FA-on, 721 vs 8893

A CLAIM WE HAD TO GET RIGHT

Compaction does not shrink the allocation. It shrinks what is read.

Llama-3.2-1B · phone GPU (Adreno 840) · 9737-token prompt

Measured	vanilla	µKV	change
Peak cache ALLOCATED	432 MB	392 MB	−9%
Cache READ per token	**304 MB**	**23 MB**	**−93%**

WHAT COMPACTION IS

Selection marks which cells to keep. Compaction slides those survivors into one contiguous block at the front of the cache, inside tensors the engine already holds.

The engine reserves the whole context up front, so peak memory barely moves. What changes is how much of it the device touches on every decoded token, and that is where the speed and the energy come from. New since: because compaction leaves the dead cells contiguous, madvise hands 485.8

WHY COMPACTION IS STILL NECESSARY

If the survivors stay scattered, the engine still walks the whole array every token. Packing them into one contiguous block is what turns 'dropped' into 'never read again'.

AND IT IS SAFE

23 of 25

generations byte-for-byte identical to the same run without compaction, across Llama-1B, Phi-3 and Bonsai-8B, with the keep-set held identical so compaction is the only variable.

The two exceptions differ by a trailing full stop and by text after the answer. Sliding survivors keeps their positions, so the maths is unchanged.

Always read the score next to how much cache was kept

Finding a fact hidden in a long document

Llama-3.2-1B · phone GPU · 14 stimuli x 7 depths

Policy	budget	found	kept
µKV	**1024**	**14/14**	**13.0%**
vanilla	full	14/14	100%
SnapKV	1024	12/14	98.2%
StreamingLLM	2004	7/14	32.9%

µKV finds every needle while keeping one eighth of the cache. StreamingLLM keeps 2.5× more and still loses half of them, because it keeps by position, not by evidence.

Question answering (LongBench hotpotqa)

Llama-3.2-1B · phone CPU · 15 questions

Policy	F1	kept
SnapKV	40.00	89.1%
vanilla	36.00	100%
µKV	**29.79**	**13.7%**
StreamingLLM	23.57	19.5%

Compared only against policies that actually evict, µKV is ahead: +6.2 F1 over StreamingLLM on less cache. The ones above it kept 6× more.

BUT BE HONEST ABOUT THIS TABLE

With only 15 questions, one question changes the score by 6.7 points. µKV and vanilla answer 13 of 15 identically. The F1 ranking here is not a real difference; the retention column is.

The watchdog gives back whatever the workload is short of

Llama-3.2-1B · phone CPU · six back-to-back 16K generations

Run	trigger	1st gen	later	at floor	peak DDR
vanilla	off	4.97	4.99	3.3%	68.3
vanilla	47 °C	5.01	4.98	3.6%	67.2
µKV	off	23.32	23.14	8.2%	65.6
µKV	**47 °C**	**26.95**	**22.97**	**6.4%**	**66.0**
µKV	35 °C	18.00	15.95	2.4%	58.7

The watchdog steps the clock down early, before the vendor's own thermal limiter slams it to 883 MHz. At its first step it also lifts a cap the vendor had already applied, which is why the first generation gets faster rather than slower.

+15.5%

µKV, first generation. It is compute-bound, so it can use the clock it is handed.

+0.9%

vanilla, identical watchdog. It is bandwidth-bound, so the same clock buys nothing and the gain shows up as −2.4 °C instead.

+10.8%

the early trigger on the GPU, where it helps. On the CPU the same early trigger costs 37% more time. The sign flips with the processor.

RESULT · ENERGY-AWARE CONTROL

Same command line; only the battery state changes the plan

GPU · clock caps by prefill / decode phase · K held at 1024 · 4096 output tokens

lever picks at	prefill / decode MHz	prefill mJ/tok	prefill s	decode mJ/tok	decode tok/s	total J	total s
full perf. (n=3)	1200 / 1200	58	117	210	40.0	1428	221
healthy, mains (n=3)	1200 / decode 902	59	117	186	38.7	1336	224
mid (n=3)	1200 / decode 726	61	117	179	37.0	1328	229
low (n=3)	902 / 902	40	154	187	38.3	1154	263
never (n=3)	726 / 726	40	191	175	36.8	1105	303

CPU · the tier sets the cache budget K · clock untouched · 1024 output tokens

lever picks at	cache budget	prefill mJ/tok	prefill s	decode mJ/tok	decode tok/s	total J	total s
every tier (n=3)	K 1024 (chosen)	60	258	209	22.1	802	305
never (n=3)	K 512	61	257	195	24.0	789	300
never (n=3)	K 256	61	256	187	24.8	783	298

A cap on decode alone is nearly free: 6% less energy for 1.5% more time and a DDR peak 19 C lower, the best energy x time of all plans. Capping prefill too saves 19% for 19% more time; below 902 the wait grows faster than the saving. On the CPU three quarters of a request is prefill, which the cache cannot touch: the tiers land within 3%, so 1024 is the CPU balance point. The cache is the CPU lever against the full cache (62% less energy), not between tiers.

-6% energy

GPU, decode capped at 902 MHz, prefill untouched: +1.4% total time, energy x time -5%. The lever's choice at full charge.

-19% energy

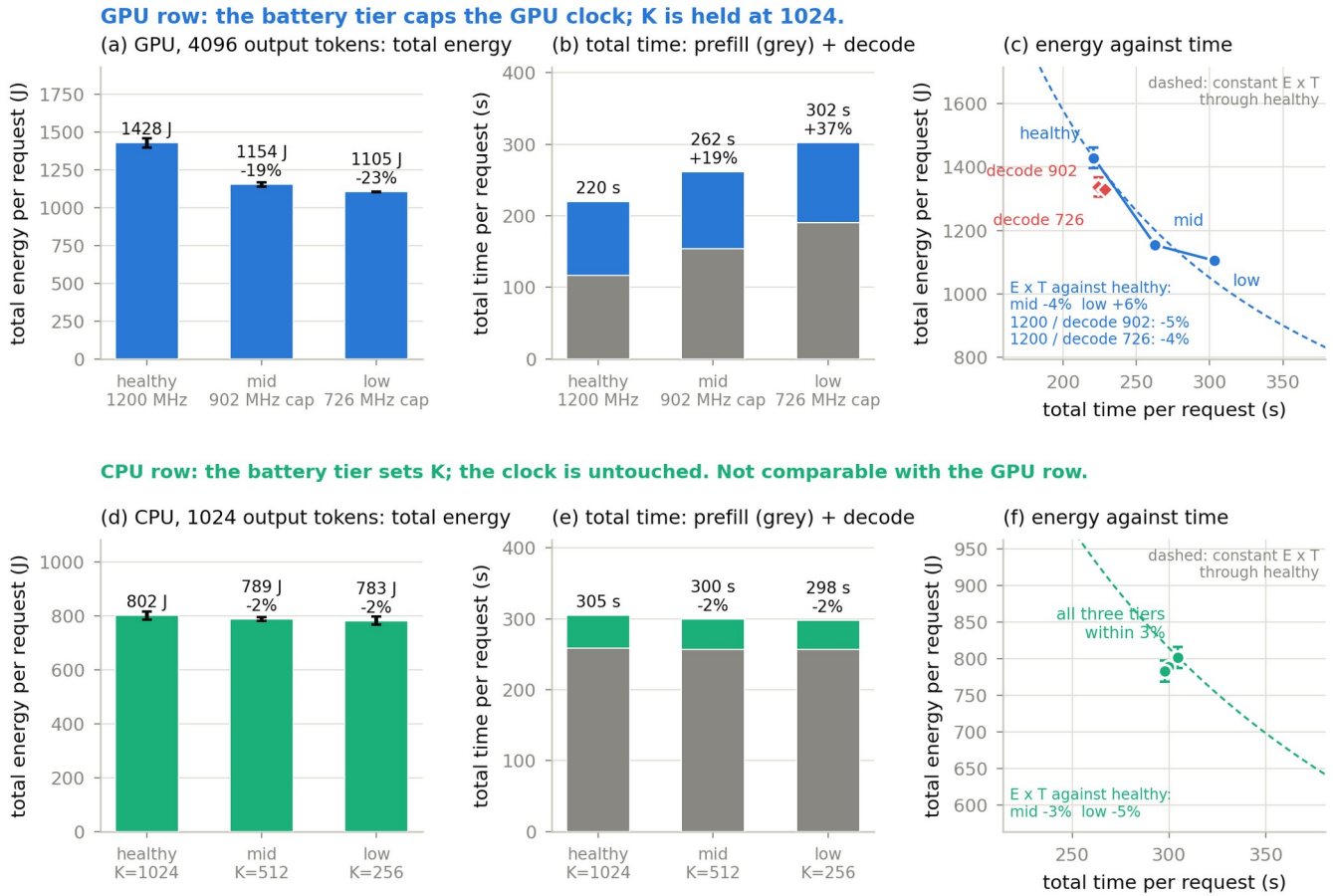
GPU, 902 MHz on both phases: +19% total time, energy x time -4%. The lever's choice at low charge; 726 is never taken.

-2% energy

CPU, K 512 instead of 1024: 37% of qasper F1 for 2% of the request. The lever holds K=1024; the cache pays against the full cache, not between tiers.

Energy falls with the tier; the price is time, not decode speed

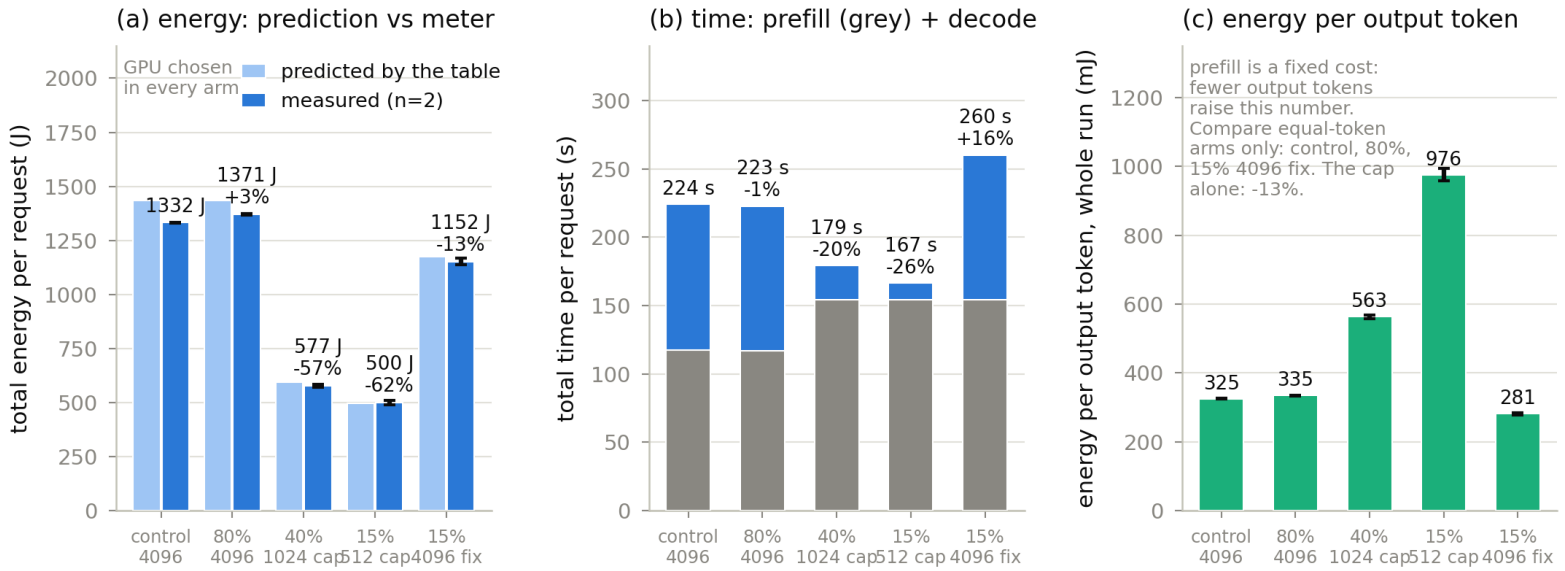
Same command line every arm, n=3. GPU row: the tier caps the clock. CPU row: the tier sets K. Rows differ in output length.



OnePlus 15, Llama-3.2-1B, 9737-token prompt. Same command line every arm; only the battery state the controller reads differs. n=3 per arm.
Tiers: healthy above 50% or mains, mid 21 to 50%, low 20% or less. CPU caps pinned, cooled per cell. Energy = USB rail + battery pack.

The scheduler chose every plan; the phone measured what it cost

Same prompt through ukv_sched.sh under forced battery states; control = told mains. Predictions from its table land within 1 to 7% of the meter.



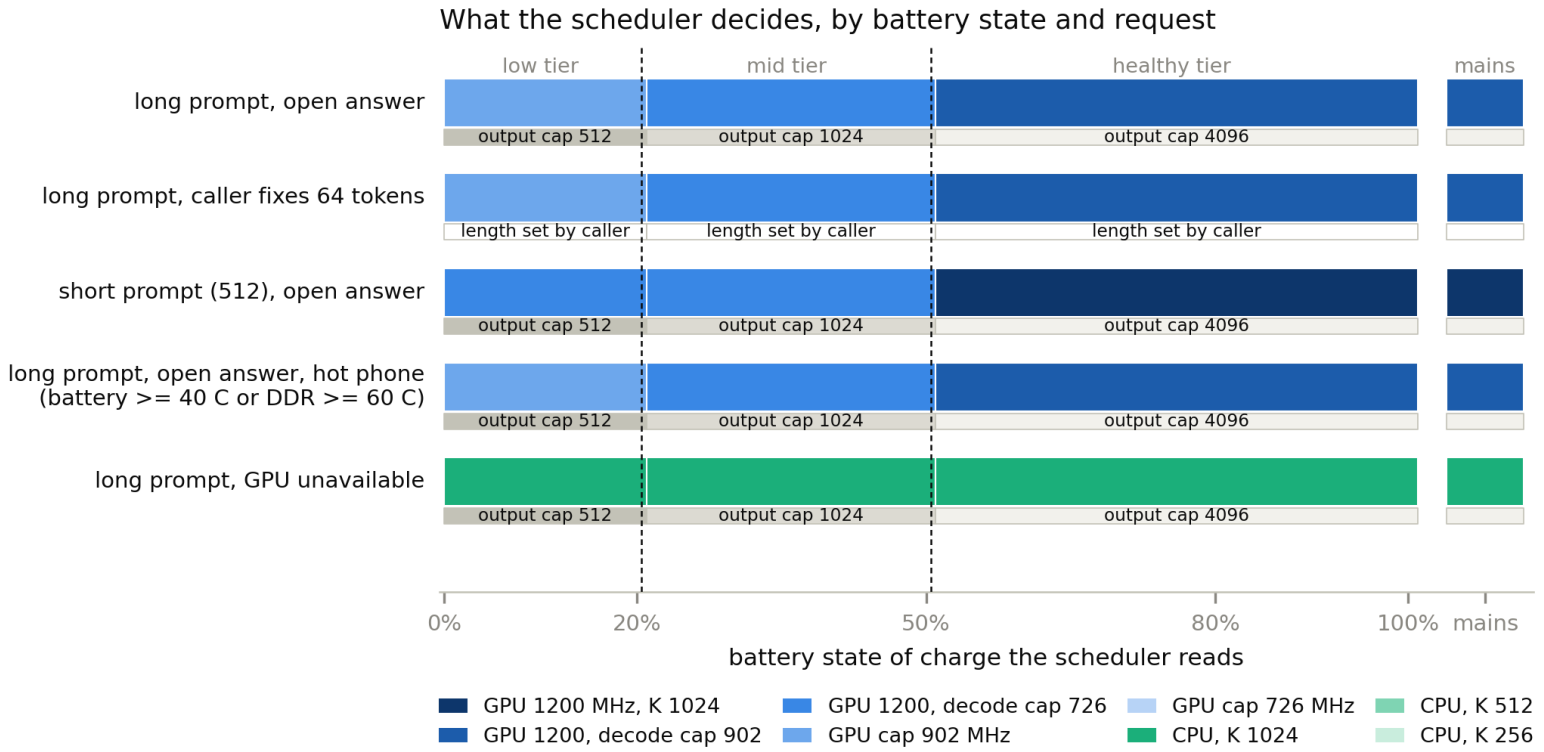
arms, as battery state told to the scheduler and the plan it chose: control = mains, GPU 1200 MHz, 4096 tokens. 80% = GPU 1200 MHz, 4096 tokens. 40% = GPU cap 902 MHz, output cap 1024. 15% = GPU cap 902 MHz, output cap 512. 15% 4096 fix = told 15%, caller fixed the length: clock cap on The CPU is chosen only when the GPU is unavailable (see the decision map).

OnePlus 15, Llama-3.2-1B, 9737-token prompt. Only the battery state was forced. n=2 per arm, CPU caps pinned, cooled per cell.

At 40% and 15% the request costs 57% and 62% less than the control, mostly through the output cap; with the output fixed by the caller (15%, 4096 fix) the clock cap alone saves 13% at equal tokens. At 80% and on mains the scheduler runs the full-performance plan.

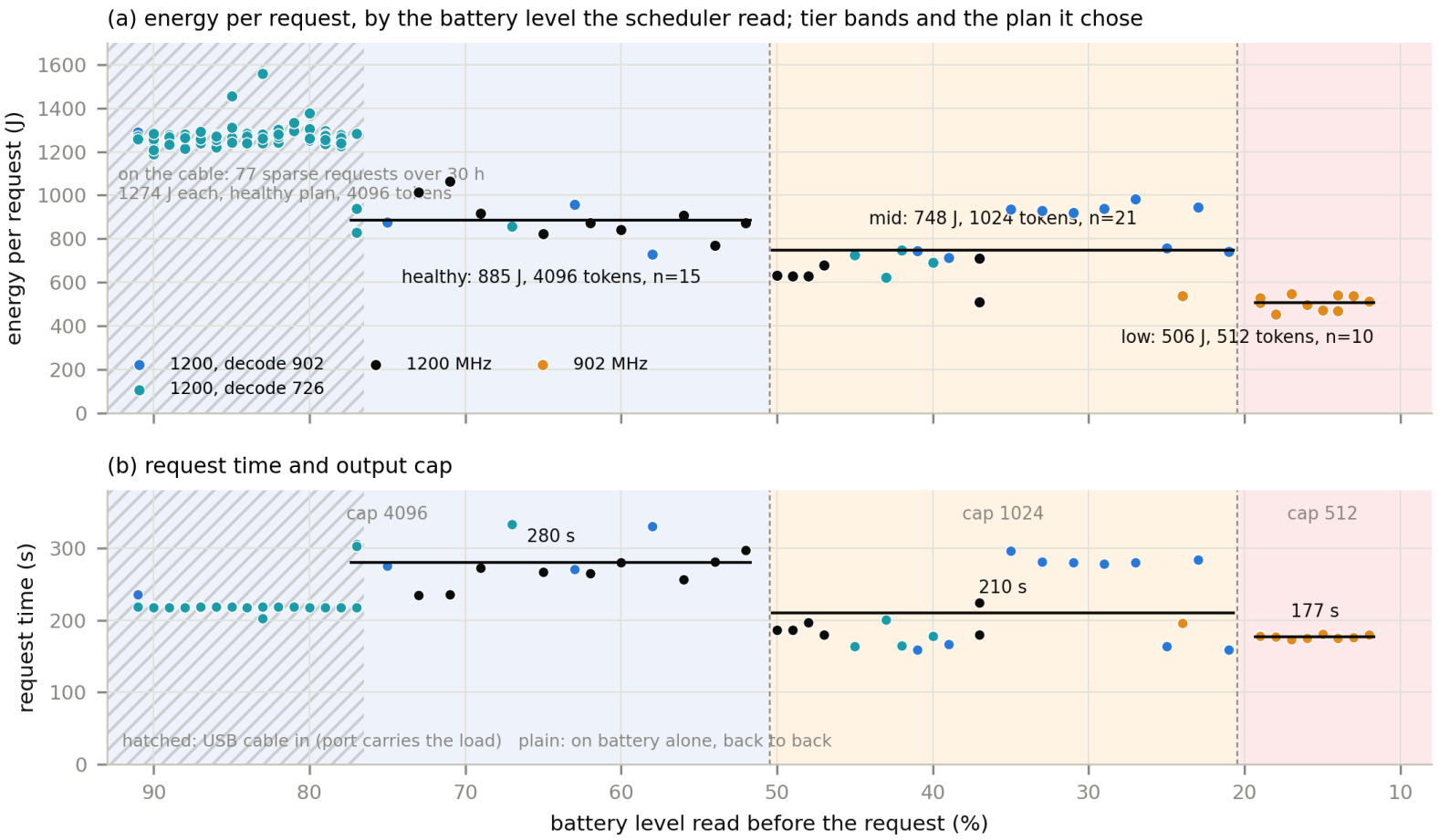
When the GPU clock, when the cache, when the output cap

The policy at every state of charge, five request situations. CPU only when the GPU is unavailable; the CPU clock is never a decision.



Upper band: backend, clock (prefill / decode), cache. Lower band: output length rule. The CPU clock is never a decision: it saves no energy on this phone. The lever (1 at mains and above 50%, 0.5 at mid, 0 when low) sets the exchange rate, the quality floor and the time budget; a step down must save at least λ times its time cost, keep quality above the floor, and fit the time budget. Split plans are predictions until measured.

RESULT · IT SWITCHES ON THE REAL BATTERY, AND SAVES



No forced state: the scheduler read the real level before every request. Charging off, cable out for the last 46 requests.

Healthy, above 50%: 4096 tokens, 885 J, 280 s
Mid, 50 to 21%: cap 1024, 748 J, 210 s (−15%)
Low, 20% and below: 902 MHz, cap 512, 506 J, 177 s (−43%)

Switches on the first request past 50 and 20.

On battery the OEM already runs the GPU cooler and caps prefill at 726 MHz within a minute, so the answer cap and the prefill cap carry the saving; the decode-clock lever (6% on the cable) is pre-empted by the vendor.

Table re-learned the battery regime in 3 requests.

OnePlus 15, Llama-3.2-1B, 9737-token prompt, --ignore-eos, no forced battery state: the scheduler read the real level before every request. Charging off; light cool gate (DDR <= 42 C) between requests; CPU caps pinned. Energies rebuilt from the raw traces with the start-anchored split and a 30 s gauge lag. Tier thresholds: healthy above 50%, mid 21 to 50%, low 20% or below.

Four contributions, each with the evidence beside it

1 **First end-to-end measurement of published eviction on a phone**

Nine policies, each at its own published setting, on four models and two processors, measured on speed, energy, temperature and cells actually freed.

Tables 1 and 2, every row re-derived from raw logs

2 **Three failure modes that belong to the device, not to any policy**

Budgets that do not materialize, a scoring route that corrupts on Adreno, and a hardware ceiling. Each is shown on published baselines, not only on ours: a problem hurting only our design would be our bug, not a finding.

Three-way control; CPU vs GPU control

3 **μKV: sequence-level, in-graph, with a characterised thermal dial**

Score in-graph, one keep-set, compact in place, freeze during decode, plus a watchdog that steps the clock before the vendor cliff. 4.80x on CPU on 7.4% of the cache, at 58% less energy.

n=3 rotated; compaction 23/25 identical

4 **The measurement traps that make phone numbers wrong**

A baseline at the wrong budget reads 5.75 instead of 29.07 tok/s. An unpinned scheduler spreads identical runs by 39%, so any GPU ratio under 1.39x is noise.

Quantified so others can avoid them

Every table in the paper was re-derived from the raw campaign data during preparation: Tables 1, 2 and 4 exact on all rows, Table 3 exact on 12/12 retention and 14/14 F1.

What is ours, and what we explicitly do not claim

The claim	Closest prior work	Our position
Evicting while FlashAttention stays on	KeyDiff does this too	No priority claimed. Our difference is per-token cost: 2,510 decode evictions vs 59,238, a 24x gap
Per-head budgets do not materialize	KV-Compress notes it qualitatively, for server tensor layouts	We quantify it on-device and show it scales with head count: 8 heads spread 36 to 91%, 32 heads collapse to 85 to 100%
Eviction measured on a phone	KeyDiff reports scoring-kernel latency only	First end-to-end account: speed, energy, temperature, and cells actually freed
Energy and thermals on mobile	Characterization studies measure them but never vary the eviction policy	We vary the policy and measure both, and add a governor characterised as a measured dial
KV systems that ship on phones	KVSwap, DynaKV, MobiLoRA move bytes to disk or across adapters	None of them evicts, and none reads the battery

WHERE THE NOVELTY ACTUALLY SITS

Not in the eviction rule. It sits in the measurement, in naming three failures that belong to the device rather than to any policy, and in composing eviction with a thermal governor, which no KV paper does.

Reported rather than quietly fixed

Cache moves heat only where it moves traffic

Halving the cache at a held GPU clock raises peak DDR 5.8 °C (mean 2.3), because the freed bandwidth becomes work. But on the CPU, where both arms settle at the same clock (1374 vs 1378 MHz), μKV is 2.4 °C COOLER on DDR at 4.6x the throughput. The sign is set by the backend, not by the cache.

Peak memory does not fall

The engine reserves the full context up front and we select only at the end of prefill, so peak allocation drops 9% while bytes read per token drop 93%. The win is bandwidth, not footprint.

Two of four models cannot run this on the GPU

Bonsai-8B and gemma-2 crash the Adreno driver at 16K with ErrorDeviceLost, vanilla included. Bonsai is also killed for memory. This is a device limit, not a μKV limit.

And the one a reviewer will find: our quality table cannot pick a winner

On 15 questions, μKV and the full cache answer 13 identically, with one win and one loss each way. A sign test gives p = 1.00, meaning no difference at all was demonstrated. The uncertainty on every score is larger than every gap in the table. Retention separates these policies. F1 at this sample size does not, and we say so rather than letting the ranking imply otherwise.

THE ONE-SENTENCE VERSION

The hardware sets the ceiling on what eviction can pay. The policy decides how much of that ceiling you capture, or whether you go backwards.

On one processor the policy spans 4.5x (1.07 to 4.80), so the policy matters. For one policy on one model, changing only the processor moves KeyDiff from 3.29x to about 0.06x, so the device matters just as much. A speedup quoted without naming both is not describing a phone.

4.80×

phone CPU, sustained decode, three runs each

7.4%

of the cache still live, and genuinely freed

+0.7%

prefill vs no eviction at a fixed clock

μKV is what the device's constraints leave you with: score inside the FlashAttention graph, select one keep-set at sequence level, pack the survivors in place, and keep the thermal loop away from the cache entirely.